

Universidad de Costa Rica  
Facultad de Ingeniería  
Escuela de Ingeniería Eléctrica

IE-0417 Diseño de Software para Ingeniería

*Profesor Rafael Esteban Badilla Alvarado – I Ciclo 2026*

---

Proyecto – Auditoría de Diseño de Software  
Entrega 1: Levantamiento del Sistema y Diagnóstico Inicial

*Sistema EIEInfo – Escuela de Ingeniería Eléctrica, UCR*

---

Mariana Bernal Hernández – C31121

Ciudad Universitaria Rodrigo Facio, Costa Rica

Junio, 2026

Repositorio auditado: <https://git.ucr.ac.cr/eieinfo/EIEInfo>

## Índice

---

|                                                   |           |
|---------------------------------------------------|-----------|
| <b>1. Resumen Ejecutivo</b>                       | <b>2</b>  |
| <b>2. Alcance y Metodología</b>                   | <b>2</b>  |
| <b>3. Ficha Técnica del Sistema</b>               | <b>2</b>  |
| <b>4. Mapa General del Sistema</b>                | <b>3</b>  |
| 4.1. Módulos activos . . . . .                    | 3         |
| 4.2. Puntos de entrada . . . . .                  | 4         |
| 4.3. Diagrama de contexto . . . . .               | 5         |
| 4.4. Diagrama de módulos y capas . . . . .        | 5         |
| <b>5. Inventario Funcional</b>                    | <b>5</b>  |
| 5.1. Actores del sistema . . . . .                | 5         |
| 5.2. Flujos principales visibles . . . . .        | 5         |
| 5.3. Módulos críticos para la operación . . . . . | 6         |
| <b>6. Hallazgos Iniciales</b>                     | <b>6</b>  |
| <b>7. Matriz Preliminar de Riesgos</b>            | <b>10</b> |
| <b>8. Plan de Acción Sugerido</b>                 | <b>11</b> |
| 8.1. Fase 1: contención inmediata . . . . .       | 11        |
| 8.2. Fase 2: estabilización . . . . .             | 11        |
| 8.3. Fase 3: refactorización controlada . . . . . | 11        |
| <b>9. Respuesta a las Preguntas Orientadoras</b>  | <b>11</b> |
| <b>10. Conclusiones</b>                           | <b>12</b> |
| <b>Anexos</b>                                     | <b>13</b> |

## Resumen Ejecutivo

---

EIEInfo es un sistema institucional de alcance considerable para la Escuela de Ingeniería Eléctrica de la Universidad de Costa Rica. El sistema concentra procesos académicos, administrativos, de egresados, inventario, firma digital, proyectos, cursos, noticias y servicios públicos en una arquitectura monolítica Django con múltiples aplicaciones internas.

El diagnóstico inicial identifica una base funcional amplia, pero con señales claras de crecimiento orgánico: módulos con responsabilidades acumuladas, lógica duplicada, dependencias desactualizadas, configuración sensible dentro del repositorio y un esquema de autenticación distribuido por portales. Los riesgos más urgentes no son de estilo, sino de seguridad operativa y mantenibilidad: credenciales versionadas, `DEBUG=True` como valor por defecto, uso de sesiones paralelas por rol y dependencias críticas sin fijación estable.

La recomendación general es priorizar primero las mitigaciones de seguridad y configuración, luego estabilizar la autenticación y finalmente abordar la deuda estructural de los módulos más acoplados. Esta entrega no propone todavía una reestructuración completa, sino una línea base verificable para decidir qué áreas deben estudiarse con más profundidad en las siguientes etapas.

## Alcance y Metodología

---

El análisis se realizó mediante inspección estática del repositorio, revisión de configuración, estructura de aplicaciones Django, rutas principales, dependencias, pruebas y organización funcional visible. No se ejecutaron pruebas dinámicas, pruebas de penetración ni despliegues locales completos; por tanto, los hallazgos se clasifican como diagnóstico preliminar y deben confirmarse con ejecución controlada cuando aplique.

- **Alcance incluido:** estructura del proyecto, configuración Django, dependencias, rutas raíz, distribución de módulos, pruebas visibles y señales de deuda técnica.
- **Alcance no incluido:** explotación de vulnerabilidades, análisis de base de datos en producción, revisión completa de permisos reales por usuario y validación funcional end-to-end.
- **Criterio de priorización:** severidad del impacto, probabilidad de ocurrencia, facilidad de explotación o aparición del problema y costo esperado de corrección.

## Ficha Técnica del Sistema

---

EIEInfo es un sistema de información institucional de la Escuela de Ingeniería Eléctrica de la Universidad de Costa Rica. Gestiona procesos académicos, administrativos y de

egresados bajo una arquitectura monolítica Django con múltiples portales diferenciados por tipo de usuario.

Cuadro 1: Ficha técnica del sistema EIEInfo

| Atributo                      | Valor                                                                                                                   |
|-------------------------------|-------------------------------------------------------------------------------------------------------------------------|
| <b>Framework principal</b>    | Django 4.1.3 con Python                                                                                                 |
| <b>Versión Python</b>         | Fijada en <code>.python-version</code>                                                                                  |
| <b>Base de datos</b>          | MariaDB / MySQL, cliente <code>mysqlclient 2.1.1</code>                                                                 |
| <b>Servidor web</b>           | Nginx como proxy inverso + Gunicorn con 3 workers, puerto 8001                                                          |
| <b>Contenedores</b>           | Docker Compose v3.8: servicios <code>db</code> , <code>nginx</code> , <code>eieinfo_app</code>                          |
| <b>CI/CD</b>                  | Drone CI, archivo <code>.drone.yml</code> en raíz del repositorio                                                       |
| <b>Tipo de arquitectura</b>   | Monolito modular Django, patrón MVT                                                                                     |
| <b>Portales de usuario</b>    | 4 portales principales: estudiantes, profesores, administrativos y alumni                                               |
| <b>Integraciones externas</b> | Google API, Facebook SDK, Instagram, reCAPTCHA, SMTP UCR, Imgur                                                         |
| <b>Wiki integrado</b>         | <code>django-wiki</code> desde fork personal en GitHub, rama <code>main</code> , sin versión fija                       |
| <b>Editores de contenido</b>  | CKEditor 6.5.1 + Martor                                                                                                 |
| <b>Tareas programadas</b>     | <code>django-crontab</code> , 3 cron jobs activos identificados                                                         |
| <b>Pruebas</b>                | <code>pytest</code> + <code>pytest-django</code> + <code>coverage</code>                                                |
| <b>Linters</b>                | <code>flake8 3.5.0</code> , versión antigua                                                                             |
| <b>Exportación de datos</b>   | <code>openpyxl</code> , <code>XlsxWriter</code> , <code>PyPDF2</code> , <code>pyexcel-xls</code> , <code>vobject</code> |
| <b>Hosting conocido</b>       | Servidor <code>faraday</code> , dominios <code>info.eie.ucr.ac.cr</code> y <code>eie.ucr.ac.cr</code>                   |

## Mapa General del Sistema

### Módulos activos

Cuadro 2: Inventario de módulos del sistema

| Módulo           | Tipo    | Responsabilidad principal                                        | Líneas |
|------------------|---------|------------------------------------------------------------------|--------|
| profesores       | Portal  | Cursos, asistencias, publicaciones, laboratorios, comisiones     | 5691   |
| administrativos  | Portal  | Noticias, reservaciones, inventario, consejo asesor, estudiantes | 4080   |
| estudiantes      | Portal  | Cursos, asistencias, trámites, bodega, práctica profesional      | 3495   |
| trabajos_finales | Dominio | Proyectos eléctricos, avances, presentaciones                    | 2485   |

| Módulo                      | Tipo    | Responsabilidad principal                                   | Líneas |
|-----------------------------|---------|-------------------------------------------------------------|--------|
| trabajo_final_de_graduacion | Dominio | Flujo nuevo de TFG, paralelo al módulo anterior             | 2175   |
| cursos                      | Dominio | Cursos, grupos, cátedras, plan de estudios                  | 1726   |
| inventario                  | Soporte | Bienes, activos, computadoras, peculios, préstamos          | 1532   |
| atributos                   | Soporte | Descriptores de cursos para acreditación                    | 1101   |
| firma_digital               | Soporte | Firma digital de documentos y notificaciones                | 964    |
| postulaciones               | Dominio | Postulaciones a asistencias, estudios formales, experiencia | 915    |
| alumni                      | Portal  | Egresados, bolsa de empleo, ofertas laborales               | 827    |
| webpage                     | Público | Página institucional pública de la EIE                      | 783    |
| eventos                     | Soporte | Gestión de eventos institucionales                          | 536    |
| anuncios                    | Soporte | Publicación de anuncios y frases                            | 511    |
| proyectos                   | Dominio | Proyectos de investigación y publicaciones                  | 483    |
| conferencias                | Dominio | Registro de conferencias y cuotas                           | 423    |
| laboratorios                | Soporte | Gestión de laboratorios por ciclo                           | 266    |

## Puntos de entrada

El archivo `src/server/eieinfo/urls.py` centraliza las rutas raíz del sistema. Se identificaron los siguientes puntos de entrada principales:

Cuadro 3: Rutas raíz del sistema

| Ruta                         | App destino                 | Descripción            |
|------------------------------|-----------------------------|------------------------|
| /estudiantes/                | estudiantes.urls            | Portal de estudiantes  |
| /profesores/                 | profesores.urls             | Portal de profesores   |
| /administrativos/            | administrativos.urls        | Portal administrativo  |
| /alumni/                     | alumni.urls                 | Portal de egresados    |
| /firma_digital/              | firma_digital.urls          | Flujo de firma digital |
| /trabajo_final_de_graduacion | trabajo_final_de_graduacion | TFG nuevo              |
| /wiki/                       | django-wiki                 | Wiki institucional     |
| /                            | webpage.urls                | Página pública         |

**Observación técnica:** se reporta que `webpage.urls` aparece como patrón amplio antes de algunas rutas específicas. Esto debe validarse mediante pruebas de resolución de

URL, porque en Django el riesgo real depende de los patrones internos incluidos por `webpage.urls`. Aun así, el orden merece revisión porque los catch-all colocados antes de rutas específicas pueden introducir errores difíciles de detectar.

## Diagrama de contexto

Ver Figura 1 en los anexos.

## Diagrama de módulos y capas

Ver Figura 2 en los anexos. La arquitectura se puede leer en cuatro capas:

- **Portales:** estudiantes, profesores, administrativos, alumni.
- **Dominio:** cursos, trabajos\_finales, trabajo\_final\_de\_graduacion, postulaciones, proyectos, conferencias, inventario, laboratorios.
- **Soporte:** anuncios, eventos, atributos, firma\_digital, webpage.
- **Núcleo:** configuración, URLs raíz, integraciones externas y manejo de credenciales.

## Inventario Funcional

---

### Actores del sistema

Cuadro 4: Actores del sistema EIEInfo

| Actor               | Capacidades principales                                                                                          |
|---------------------|------------------------------------------------------------------------------------------------------------------|
| Estudiante          | Cursos y calificaciones, asistencias, TFG, trámites, bodega, práctica profesional, proyecto eléctrico            |
| Profesor            | Cursos, asistencias, publicaciones, conferencias, laboratorios, TFG asignados, comisiones, noticias              |
| Administrativo      | Noticias, reservaciones, inventario, nombramientos, consejo asesor, gestión de estudiantes, práctica profesional |
| Alumni              | Red de egresados, bolsa de empleo, recepción de ofertas laborales por correo                                     |
| Visitante público   | Página institucional EIE, información pública de cursos y noticias                                               |
| Director de Escuela | Notificaciones de firma digital y aprobación de documentos formales                                              |
| Sistema / cron      | Divulgación de ofertas, recordatorios de asistencias y notificaciones periódicas                                 |

### Flujos principales visibles

Cuadro 5: Flujos funcionales principales

| Flujo                       | Portales                                 | Descripción                                                                             |
|-----------------------------|------------------------------------------|-----------------------------------------------------------------------------------------|
| Gestión de asistencias      | Profesores, Estudiantes, Administrativos | Registro, consulta y control de asistencias por curso, laboratorio, comisión y proyecto |
| Trabajo Final de Graduación | Estudiantes, Profesores                  | Creación, seguimiento, revisión y defensa de TFG                                        |
| Proyecto Eléctrico          | Estudiantes, Profesores                  | Avance, prórroga y presentación de proyecto eléctrico                                   |
| Práctica profesional        | Estudiantes, Administrativos             | Solicitud, revisión, cartas, firma y finalización                                       |
| Consejo Asesor              | Profesores, Administrativos              | Ciclos, grupos, cátedras, nombramientos y comisiones                                    |
| Bolsa de empleo             | Alumni, Profesores                       | Publicación y envío periódico de ofertas a egresados                                    |
| Firma digital               | Profesores, Director                     | Carga, firma y notificación de documentos                                               |
| Gestión de inventario       | Administrativos, Profesores              | Control de bienes, activos, computadoras, peculios y préstamos                          |

### Módulos críticos para la operación

- **profesores y estudiantes:** portales con mayor volumen funcional y alta exposición a usuarios internos.
- **cursos:** modelo de dominio central, referenciado por múltiples módulos.
- **trabajo\_final\_de\_graduacion y trabajos\_finales:** posible duplicación del dominio de trabajos finales.
- **firma\_digital:** flujo formal con impacto institucional.
- **administrativos:** concentra varias áreas funcionales, por lo que representa un punto de acoplamiento importante.
- **eieinfo / configuración:** concentra settings, rutas raíz, credenciales e integraciones externas.

### Hallazgos Iniciales

Se presentan 15 hallazgos identificados a partir del análisis estático del repositorio. Cada hallazgo incluye evidencia, impacto y una recomendación inicial. La criticidad debe interpretarse como priorización preliminar, no como resultado de una prueba de explotación.

### H-01 – DEBUG=True como valor por defecto

**Descripción:** El archivo `settings.py` define `DEBUG = True` y solo lo desactiva si el hostname coincide con `faraday`. Cualquier ambiente con otro hostname podría ejecutarse en modo debug.

**Evidencia:** `docker/django/settings.py`: definición inicial de `DEBUG = True` y bloque condicional dependiente de `socket.gethostname()`.

**Impacto:** Puede exponer stack traces, rutas internas, configuración sensible y detalles de ejecución ante errores en ambientes no controlados.

**Recomendación inicial:** Mover `DEBUG` a una variable de entorno obligatoria, usar `False` por defecto y ejecutar `python manage.py check -deploy` como control de despliegue.

### H-02 – Credenciales sensibles versionadas

**Descripción:** El repositorio contiene un archivo de credenciales importado desde la configuración. Esto expone secretos a toda persona con acceso al repositorio.

**Evidencia:** `docker/django/secret_credentials.py` y uso desde `settings.py`.

**Impacto:** Compromiso potencial de claves API, contraseñas, tokens de servicios externos y secreto de Django.

**Recomendación inicial:** Eliminar secretos del repositorio, rotar credenciales, usar variables de entorno o un gestor de secretos y agregar reglas de prevención en CI.

### H-03 – Contraseña SMTP en configuración

**Descripción:** La configuración de correo contiene una contraseña definida directamente como literal o valor estático.

**Evidencia:** `docker/django/settings.py`, sección de correo, `EMAIL_HOST_PASSWORD`.

**Impacto:** Riesgo de uso no autorizado del servidor SMTP institucional y exposición de credenciales por control de versiones.

**Recomendación inicial:** Sustituir el valor por una variable de entorno, rotar la contraseña y revisar historial de Git para confirmar si el secreto fue expuesto previamente.

### H-04 – Autenticación híbrida con sesiones paralelas

**Descripción:** Cada portal utiliza claves de sesión propias, como `est_id`, `prof_id`, `adm_n_id` y `alumnus_id`. Además, algunos permisos parecen cachearse en sesión.

**Evidencia:** `estudiantes/misc.py`, `profesores/misc.py`, `administrativos/misc.py`, `alumni/misc.py`.

**Impacto:** La política de acceso queda distribuida y es más difícil garantizar revocación inmediata, consistencia entre portales y trazabilidad de permisos.

**Recomendación inicial:** Centralizar autenticación con `django.contrib.auth`, grupos/permisos de Django y decoradores comunes. Como paso intermedio, documentar todos los permisos de sesión y añadir pruebas de autorización por portal.



**H-05 – ALLOWED\_HOSTS permisivo fuera de producción**

**Descripción:** La configuración permite `ALLOWED_HOSTS = ['*']` cuando el host-name no coincide con el ambiente esperado.

**Evidencia:** `settings.py`, bloque condicional final.

**Impacto:** Puede habilitar ataques relacionados con Host header en ambientes de staging, pruebas compartidas o despliegues mal configurados.

**Recomendación inicial:** Definir `ALLOWED_HOSTS` mediante variable de entorno y fallar el arranque si no se configura explícitamente en ambientes no locales.

**H-06 – God Object en consejo\_asesor.py**

**Descripción:** El archivo `profesores/views/consejo_asesor.py` concentra una cantidad alta de funciones y responsabilidades: ciclos, grupos, cátedras, nombramientos, laboratorios, proyectos y reportes.

**Evidencia:** `profesores/views/consejo_asesor.py`, aproximadamente 1810 líneas.

**Impacto:** Aumenta el riesgo de regresiones, dificulta pruebas unitarias y vuelve costoso modificar una funcionalidad sin afectar otras.

**Recomendación inicial:** Dividir por casos de uso o subdominios, mover lógica de negocio a servicios y crear pruebas de caracterización antes de refactorizar.

**H-07 – Duplicación del dominio TFG**

**Descripción:** Existen dos módulos relacionados con trabajos finales: `trabajos_finales` y `trabajo_final_de_graduacion`. Esto sugiere transición incompleta o coexistencia de modelos de dominio.

**Evidencia:** `trabajos_finales/models.py`, `trabajo_final_de_graduacion/models.py` y rutas comentadas en portales.

**Impacto:** Puede generar ambigüedad conceptual, duplicación de reglas de negocio e inconsistencia de datos.

**Recomendación inicial:** Definir cuál módulo es canónico, documentar el estado del módulo legado y crear un plan de migración o retiro controlado.

**H-08 – Módulo administrativos con responsabilidades múltiples**

**Descripción:** El módulo `administrativos` concentra vistas para asistencias, consejo asesor, estudiantes, inventario, noticias, práctica profesional, reservaciones y home.

**Evidencia:** `administrativos/views/`: múltiples archivos funcionalmente distintos dentro de un mismo portal.

**Impacto:** Baja cohesión del módulo y dificultad para evolucionar funcionalidades administrativas de manera independiente.

**Recomendación inicial:** Separar responsabilidades por submódulos o apps cuando el dominio lo justifique; inicialmente, documentar fronteras y añadir pruebas por flujo.

### H-09 – Fork de django-wiki sin versión fija

**Descripción:** La dependencia del wiki se resuelve directamente desde la rama `main` de un fork personal en GitHub.

**Evidencia:** `requirements.txt`: dependencia `git+https://.../django-wiki@main`.

**Impacto:** Un cambio posterior en la rama puede romper instalaciones futuras o introducir comportamiento no revisado.

**Recomendación inicial:** Fijar la dependencia a un commit específico, tag o paquete versionado; documentar por qué se usa el fork y quién lo mantiene.

### H-10 – trabajos\_finales sin pruebas activas

**Descripción:** El módulo de proyectos eléctricos tiene su archivo de pruebas vacío o comentado pese a su tamaño funcional.

**Evidencia:** `trabajos_finales/tests.py`: contenido mínimo o comentado.

**Impacto:** No existe red de seguridad automatizada para cambios en un módulo funcionalmente relevante.

**Recomendación inicial:** Agregar pruebas de caracterización sobre flujos existentes antes de cualquier refactorización o migración del módulo.

### H-11 – Módulos incompletos o zombi

**Descripción:** Existen directorios que parecen apps o restos de apps, pero no contienen implementación activa suficiente.

**Evidencia:** `asistencias/`, `practica_profesional/`, `proyecto_electrico/`.

**Impacto:** Confunde límites del sistema y puede hacer más costosa la incorporación de nuevos desarrolladores.

**Recomendación inicial:** Clasificar cada directorio como activo, legado o eliminable. Si es legado, documentarlo; si no se usa, removerlo en una rama controlada.

### H-12 – Zona horaria semánticamente incorrecta

**Descripción:** La configuración usa `America/Guatemala` aunque el sistema opera en Costa Rica. Aunque ambas zonas suelen compartir UTC-6, la configuración es conceptualmente incorrecta para un sistema institucional costarricense.

**Evidencia:** `settings.py`: `TIME_ZONE = 'America/Guatemala'`.

**Impacto:** El impacto técnico puede ser limitado si el offset coincide, pero afecta claridad operativa, auditoría y mantenibilidad.

**Recomendación inicial:** Cambiar a `America/Costa_Rica` y verificar cron jobs, reportes y timestamps históricos antes de desplegar.

### H-13 – Templates duplicados entre módulos

**Descripción:** Se identifican plantillas HTML duplicadas o muy similares entre ubicaciones distintas.

**Evidencia:** `form_curso.html`, `form_proyecto.html`, `info_grupo.html`, `info_horario.html`.

**Impacto:** Una corrección visual o funcional debe replicarse en varios archivos, aumentando riesgo de divergencia.

**Recomendación inicial:** Extraer parciales reutilizables, componentes de plantilla o herencia común. Priorizar los templates usados por más de un portal.

#### H-14 – Dependencias desactualizadas

**Descripción:** Varias dependencias corresponden a versiones antiguas o bibliotecas deprecadas.

**Evidencia:** `requirements.txt`: `flake8==3.5.0`,  
`google-api-python-client==1.5.3`, `oauth2client==3.0.0`,  
`facebook-sdk==2.0.0`, `PyPDF2==1.26.0`.

**Impacto:** Puede haber vulnerabilidades conocidas, incompatibilidades futuras y mayor dificultad para actualizar Python o Django.

**Recomendación inicial:** Ejecutar auditoría de dependencias, actualizar primero bibliotecas de seguridad y reemplazar paquetes deprecados como `oauth2client`.

#### H-15 – Encoding UTF-8 corrupto en configuración

**Descripción:** Se observan nombres con caracteres corruptos en la configuración, por ejemplo en contactos de `MANAGERS`.

**Evidencia:** `settings.py`, lista `MANAGERS`.

**Impacto:** Puede afectar correos automáticos, legibilidad de logs y calidad general del código fuente.

**Recomendación inicial:** Normalizar archivos a UTF-8, corregir literales afectados y añadir revisión de encoding en el proceso de mantenimiento.

## Matriz Preliminar de Riesgos

Cuadro 6: Matriz preliminar de riesgos

| Riesgo                                    | Causa      | Impacto | Prob. | Prior. | Horizonte   |
|-------------------------------------------|------------|---------|-------|--------|-------------|
| Exposición de credenciales                | H-02, H-03 | Crítico | Alta  | P1     | Inmediato   |
| Información sensible por debug            | H-01       | Crítico | Media | P1     | Inmediato   |
| Autorización inconsistente entre portales | H-04       | Alto    | Media | P2     | Corto plazo |
| Ruptura por dependencia no fijada         | H-09       | Alto    | Alta  | P2     | Corto plazo |
| Inconsistencia entre módulos TFG          | H-07       | Alto    | Media | P2     | Corto plazo |

| Riesgo                                    | Causa | Impacto | Prob. | Prior. | Horizonte     |
|-------------------------------------------|-------|---------|-------|--------|---------------|
| Cambios riesgosos en consejo asesor       | H-06  | Alto    | Alta  | P2     | Corto plazo   |
| Cambios sin cobertura en trabajos finales | H-10  | Medio   | Alta  | P3     | Mediano plazo |
| Dependencias obsoletas                    | H-14  | Medio   | Alta  | P3     | Mediano plazo |
| Divergencia de templates duplicados       | H-13  | Medio   | Media | P3     | Mediano plazo |
| Confusión por módulos zombi               | H-11  | Bajo    | Alta  | P4     | Largo plazo   |

## Plan de Acción Sugerido

---

### Fase 1: contención inmediata

1. Rotar credenciales expuestas y retirarlas del repositorio.
2. Cambiar `DEBUG` para que dependa de variable de entorno y sea `False` por defecto.
3. Definir `ALLOWED_HOSTS` de forma explícita por ambiente.
4. Fijar la dependencia del fork de `django-wiki` a un commit o versión estable.

### Fase 2: estabilización

1. Documentar el modelo de autenticación actual por portal.
2. Agregar pruebas de autorización para rutas críticas.
3. Crear pruebas de caracterización en `trabajos_finales` antes de modificarlo.
4. Confirmar estado real de módulos legados o incompletos.

### Fase 3: refactorización controlada

1. Separar responsabilidades de `consejo_asesor.py` por casos de uso.
2. Definir el módulo canónico para TFG y planear migración desde el módulo legado.
3. Extraer templates duplicados a componentes compartidos.
4. Actualizar dependencias por grupos, con pruebas de regresión.

## Respuesta a las Preguntas Orientadoras

---

Cuadro 7: Preguntas orientadoras y respuestas

| Pregunta                                           | Respuesta con evidencia                                                                                                                                                                                                                  |
|----------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ¿Qué tan centralizado o fragmentado?               | Centralizado en configuración y rutas raíz, pero fragmentado en portales. La autenticación y parte de la autorización parecen repetirse por tipo de usuario.                                                                             |
| ¿Cuáles módulos concentran más responsabilidad?    | <b>profesores</b> y <b>administrativos</b> concentran alto volumen funcional. Dentro de <b>profesores</b> , <b>consejo_asesor.py</b> aparece como el punto individual más acoplado.                                                      |
| ¿Partes activas vs. legadas?                       | Activas: <b>trabajo_final_de_graduacion</b> , <b>firma_digital</b> , <b>postulaciones</b> . Posiblemente legadas o incompletas: <b>trabajos_finales</b> , <b>asistencias</b> , <b>practica_profesional</b> , <b>proyecto_electrico</b> . |
| ¿Señales de crecimiento orgánico?                  | Rutas comentadas, módulos vacíos, templates duplicados, dependencias antiguas, encoding corrupto y configuración dependiente de hostname.                                                                                                |
| ¿Cuáles son los primeros riesgos de mantenimiento? | El archivo <b>consejo_asesor.py</b> , el módulo <b>administrativos</b> , la duplicación del dominio TFG y la autenticación por sesiones paralelas.                                                                                       |

## Conclusiones

EIEInfo es un sistema funcional y relevante para la operación institucional de la Escuela de Ingeniería Eléctrica. Su principal problema no es la ausencia de funcionalidad, sino la acumulación de deuda técnica alrededor de configuración, seguridad, autenticación y separación de responsabilidades.

Los hallazgos más urgentes son las credenciales en código fuente, el modo debug como valor por defecto y la configuración permisiva de hosts. Estos deben atenderse antes de iniciar refactorizaciones profundas. En paralelo, la autenticación híbrida por portales debe estudiarse como problema transversal, porque afecta seguridad, mantenibilidad y evolución futura del sistema.

Desde el punto de vista de diseño, los puntos más delicados son **consejo\_asesor.py**, el módulo **administrativos** y la coexistencia de dos dominios relacionados con trabajos finales. Para las siguientes entregas, conviene profundizar en estos tres frentes mediante diagramas de dependencias, pruebas de caracterización y propuestas de refactorización gradual.

En síntesis, el sistema tiene una base técnica rescatable, pero requiere intervención priorizada. La estrategia más segura es contener primero los riesgos críticos, estabilizar pruebas y permisos, y solo después reorganizar los módulos con mayor acoplamiento.

## Anexos

---

### Anexo A – Diagrama de Contexto

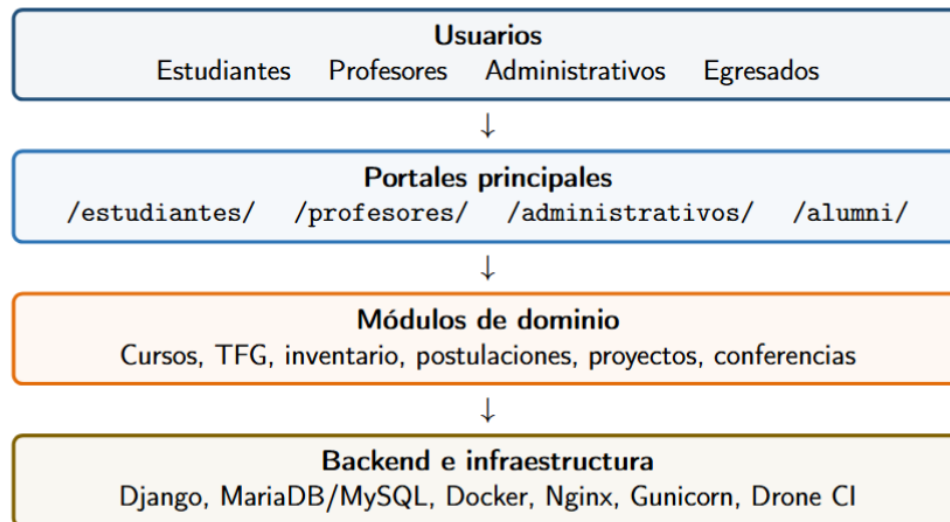


Figura 1: Diagrama de contexto del sistema EIEInfo

### Anexo B – Diagrama de Módulos por Capas

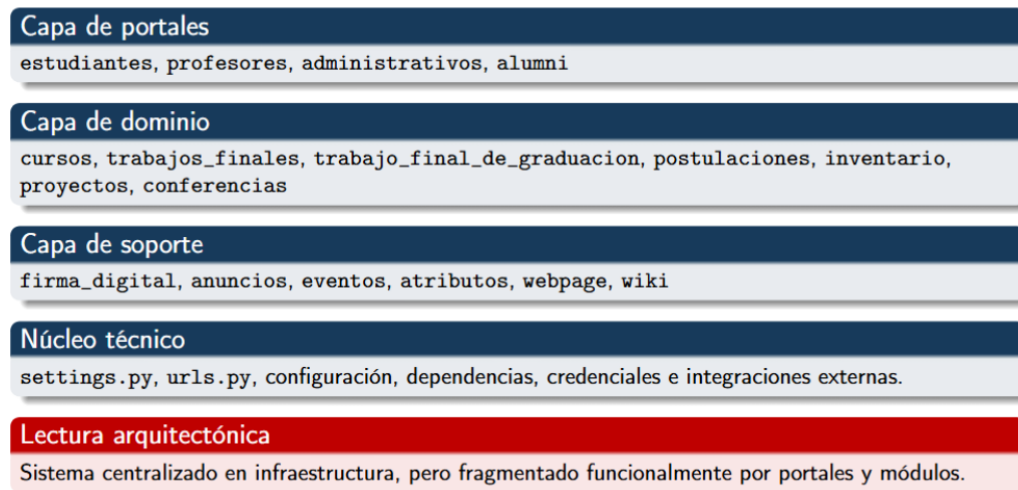


Figura 2: Diagrama de módulos y capas del sistema EIEInfo

## Anexo C – Cron Jobs Activos

| Horario              | Función                    | Módulo             |
|----------------------|----------------------------|--------------------|
| 0 8 * * * diario 8am | DivulgarOfertasEmpleo      | alumni.cron        |
| 0 8 * * 1 lunes 8am  | EmailAsistenciasPendientes | profesores.cron    |
| 0 7 * * * diario 7am | notificar_director_diario  | firma_digital.cron |

Cuadro 8: Cron jobs activos del sistema

## Anexo D – Cobertura de Pruebas por Módulo

| Módulo                         | Líneas | Métodos | Estado                     |
|--------------------------------|--------|---------|----------------------------|
| firma_digital                  | 2348   | 177     | Buena cobertura relativa   |
| profesores                     | 1607   | 95      | Buena cobertura relativa   |
| asistencias                    | 830    | 7       | Tests sin app activa clara |
| administrativos                | 935    | 32      | Cobertura parcial          |
| trabajo_final_de_graduacion713 |        | 38      | Cobertura parcial          |
| webpage                        | 618    | 88      | Buena cobertura relativa   |
| postulaciones                  | 604    | 26      | Cobertura parcial          |
| estudiantes                    | 367    | 31      | Cobertura parcial          |
| proyecto_electrico             | 318    | 2       | Tests sin app activa clara |
| proyectos                      | 200    | 25      | Cobertura básica           |
| cursos                         | 181    | 15      | Cobertura básica           |
| atributos                      | 101    | 3       | Cobertura mínima           |
| alumni                         | 34     | 1       | Cobertura mínima           |
| trabajos_finales               | 3      | 0       | Sin pruebas activas        |

Cuadro 9: Cobertura de pruebas por módulo